

TP : Conception d'un ECU Temps Réel – Régulateur de Vitesse

Public : Ingénieurs Systèmes Embarqués / Temps Réel

Support : ESP32 & FreeRTOS

Durée : ~ 10 heures (Mini-projet)

Objectif

L'ECU (Electronic Control Unit) à concevoir est un régulateur de vitesse embarqué. Il reçoit des informations de vitesse et de consigne via un bus de communication, calcule une commande moteur, et la retransmet sur ce même bus. Le système doit rester opérationnel sous charge élevée et garantir un comportement sûr en cas de défaillance.

Le système doit :

- Traiter un flux de messages asynchrones via UART.
- Calculer une commande via un algorithme PID.
- Garantir le déterminisme et la sécurité (Failsafe) sous forte charge.
- Émettre des messages de contrôle et de télémétrie
- Garantir un comportement **temps réel** sous **FreeRTOS**

Contexte

Dans un véhicule moderne, un régulateur de vitesse repose sur plusieurs ECU qui échangent des informations :

- consigne de vitesse
- vitesse mesurée
- commandes moteur

Dans ce TP :

- Vous implémentez un **ECU de régulation**
- Les communications seront simulées via **UART**
- Le protocole est inspiré du CAN, mais à **taille variable**

 Le système devra fonctionner correctement **même en présence d'erreurs de communication et de surcharge**

Protocole de Communication

Le bus est simulé par une liaison UART (115200 bauds). Le protocole utilisé ici est inspiré du CAN.

Format d'une trame :

[START: 0xAA] [LEN: 2 octet] [TYPE: 1 octet] [PAYLOAD: N octets] [CRC: 1 octet]

- **LEN** : Taille de (TYPE + PAYLOAD).
- **CRC** : XOR de tous les octets de la trame sauf le START.
- **Endianness** : Little Endian pour les types **float** (IEEE754) et **uint16/32**.

Messages principaux

ID	Nom	Format Payload	Description
0x01	SETPOINT	float (4b)	Consigne de vitesse visée.
0x02	SPEED	float (4b)	Vitesse actuelle mesurée par les capteurs.
0x05	MODE_SET	uint8 (1b)	0: OFF, 1: MANUAL, 2: AUTO (PID).
0x80	OUTPUT	float (4b)	Commande moteur calculée (Sortie ECU).
0x83	STATS	uint32 * N	Télémétrie: envoi de compteurs : RX valide, RX corruption, RX dropped, TX...
0x85	ALARM	string	Message d'erreur critique
0xff	DBG	string	Message de debug

Toute trame dont le CRC est invalide, dont la longueur est incohérente, ou dont l'ID est inconnu doit être rejetée silencieusement — sans interrompre le traitement des trames suivantes. Un compteur d'erreur pourra être renvoyé via la télémétrie.

Modes de fonctionnement

L'ECU opère selon trois modes sélectionnables via le message **MODE_SET** :

Mode OFF (0x00) — L'ECU est inactif. La commande moteur est maintenue à 0. Aucun calcul n'est effectué.

Mode MANUAL (0x01) — La commande moteur est maintenue à 0. Réservé à un usage futur ou à des tests externes.

Mode AUTO (0x02) — L'ECU active le régulateur. Il calcule et émet une commande moteur (**OUTPUT**) de façon périodique, à partir de la consigne **SETPoint** et de la vitesse mesurée **SPEED**.

Un changement de mode doit être pris en compte immédiatement. Lors d'une transition vers un mode autre qu'AUTO, la commande moteur doit être remise à 0.

Exigences fonctionnelles

01 — Régulation périodique. En mode AUTO, l'ECU calcule et émet un message **OUTPUT** toutes les 100 ms. Cette période est une contrainte stricte : le jitter toléré est inférieur à 5 ms.

02 — Algorithme de régulation. Le calcul de la commande repose sur un régulateur PID discret. La sortie est saturée dans l'intervalle [0, 255]. (cf section *Algorithme de Contrôle (PID)*)

03 — Robustesse du récepteur. Le récepteur doit gérer correctement les situations suivantes au des messages malformés, incomplets, invalide arriverait, sans perte d'état ni blocage.

04 — Télémétrie et Émission des statistiques. L'ECU émet périodiquement (1s) un message **STATS** contenant quatre compteurs cumulatifs depuis le démarrage :

- nombre de trames valides reçues ;
- nombre de trames rejetées pour CRC invalide ;
- nombre de trames perdues (queue pleine) ;
- nombre de messages **OUTPUT** émis.
- Tout autre statistiques jugés intéressantes laissé libre au concepteur

05 — Mécanisme de sécurité (Failsafe). Deux événements déclenchent indépendamment le failsafe :

1. Aucune trame valide reçue depuis plus de **2 secondes**.
2. Front détecté sur le **GPIO d'erreur externe** (au choix). Le temps de réponse requis est inférieur à 5 ms. On pourra utiliser une LED.

Lors du déclenchement, l'ECU doit :

- forcer la commande moteur à 0 ;
- passer en mode OFF ;
- émettre immédiatement un message ALERT (0x85) avec un texte identifiant la cause.

06 — Reprise après failsafe. L'ECU doit être capable de reprendre un fonctionnement normal en mode AUTO après un failsafe, dès réception d'un nouveau message MODE_SET.

Exigences non fonctionnelles

01 — Déterminisme. Le système doit garantir le respect de la période de régulation même sous forte charge de réception UART.

02 — Isolation des défaillances. Une anomalie sur la réception (flux corrompu, flood) ne doit pas perturber la régulation ni retarder le failsafe.

03 — Intégrité des émissions. Toute trame émise sur le bus doit être complète et cohérente. Un entrelacement d'octets appartenant à des trames différentes est interdit.

04 — Absence d'inversion de priorité non maîtrisée. L'utilisation des primitives de synchronisation doit garantir qu'aucune tâche critique n'est bloquée indéfiniment par une tâche de priorité inférieure.

Algorithme de Contrôle (PID)

En mode AUTO (0x05), l'ECU doit calculer la commande toutes les 100 ms :

$$error = setpoint - speed$$

$$output = (K_p \cdot error) + (K_i \cdot \int error dt) + (K_d \cdot \frac{d(error)}{dt})$$

- **Coefficients :**

$$K_p = 1.0, K_i = 0.1, K_d = 0.01$$

Les coefficients du PID peuvent être adaptés au besoin.

- **Contrainte :** La sortie output doit être saturée entre 0 et 255.

Tests

Votre ECU pourra être testé par un script Python injectant :

- Des trames concaténées (collées les unes aux autres).

- Des trames fragmentées (pauses au milieu d'un message).
- Des erreurs de CRC et des IDs inconnus.
- Une surcharge CPU (Flood de messages).

Travail demandé et livrables

1. **Conception** : Définir une architecture et justifier les choix. Réaliser un diagramme des tâches et justifier les priorités...
2. **Implémentation** : Développer le code (sous ESP-IDF, Arduino ou autre) avec FreeRTOS.

Livable:

- Code source du projet : Développer le code sous ESP-IDF avec FreeRTOS.
- Un rapport succinct contenant :

Architecture et Analyse Temporelle :

- Choix d'architecture (design, tâches, etc...)
- Tableau des tâches : Nom, Priorité, Périodicité, Taille de la Stack.
- Diagramme des tâches avec tous les événements possibles et justifier les priorités.
- Justification de l'utilisation des mécanismes FreeRTOS
- Calcul de la charge CPU théorique.
- Comment avez-vous géré la priorité de l'UART par rapport au contrôle moteur ?
- Choix des priorités et gestion de la préemption?
- ...

Monitoring & Debug : Vos solutions de monitoring et debug.

Analyse de robustesse : Comment vous avez testé votre équipement et les résultats obtenus face à un script de stress test.

Limitations : expliciter les limitations de l'équipement si il y'en a.



Contraintes importantes

- Une architecture monolithique sera pénalisée
- L'utilisation effective des mécanismes FreeRTOS est attendue
- La robustesse est un critère majeur d'évaluation

ANNEXES

Configuration du projet

1. Initialisation du projet

Ouvrez un terminal avec l'environnement *IDF*. Peut être lancé depuis *eim*
Placez-vous dans votre dossier de travail habituel et créez la structure du projet :

```
$ idf.py create-project ecu  
$ cd ecu
```

2. Définition de la cible matérielle

Il est crucial de dire à IDF pour quelle puce il doit compiler (on utilise un ESP32 classique) :

```
$ idf.py set-target esp32
```

3. Configuration (Optionnel)

C'est ici que vous pouvez régler la vitesse de l'horloge, les paramètres FreeRTOS ou la vitesse du port série :

```
$ idf.py menuconfig
```

*Une fois dans le menu, naviguez, faites vos changements, appuyez sur **S** pour sauvegarder et **Q** pour quitter.*

4. Compilation, Flash et Monitoring

Vous pouvez enchaîner les commandes. Le "flash" envoie le code sur la carte, et le "monitor" ouvre la console série intégrée (alternative à picocom qui permet de voir les erreurs de crash décodées) :

```
$ idf.py build  
$ idf.py -p /dev/ttyUSB0 flash monitor
```

Résumé des commandes utiles

Commande	Action
<code>idf.py build</code>	Compile uniquement le code.

idf.py flash	Téléverse le binaire sur l'ESP32.
idf.py monitor	Ouvre la console série (Quitter avec Ctrl + J).
idf.py clean	Supprime les fichiers de compilation (si bug étrange).
idf.py fullclean	Supprime tout le dossier build pour repartir de zéro.

Example implementation PID

```
#include <stdio.h>
#include <unistd.h>

int main() {
    const float Kp = 2.0f;
    const float Ki = 0.8f;
    const float Kd = 0.02f;
    const float dt = 0.1f;

    const float out_min = 0.0f;
    const float out_max = 255.0f;

    const float motor_gain = 1.65f;
    const float drag_coeff = 0.9f;
    const float model_response = 1.2f;
    const float max_speed = 220.0f;

    float integral = 0.0f;
    float setpoint = 90.0f;
    float speed = 50.0f;
    float prev_err = setpoint - speed;
    float output = 0.0f;

    for (int i = 0; i < 150; i++) {
        float err = setpoint - speed;
        float integral_candidate = integral + err * dt;

        // Terme Dérivé
        float der = (err - prev_err) / dt;

        output = (Kp * err) + (Ki * integral_candidate) + (Kd * der);

        // Saturation + anti-windup
        if (output > out_max) {
            output = out_max;
            if (err > 0.0f) {
                integral_candidate = integral;
            }
        } else if (output < out_min) {
            output = out_min;
            if (err < 0.0f) {
                integral_candidate = integral;
            }
        }

        integral = integral_candidate;
        prev_err = err;

        // Modèle physique
        float acceleration = model_response * ((motor_gain * output) - (drag_coeff *
speed));
        speed += acceleration * dt;
        if (speed < 0.0f) speed = 0.0f;
        if (speed > max_speed) speed = max_speed;
    }
}
```



```

        printf("Consigne: %0.1f | Commande: %0.2f | Vitesse: %0.2f\n",
               setpoint, output, speed);

        usleep(200000);
    }
    return 0;
}

```

Exemple script de test

```

import os
import serial
import struct
import time
import random
import threading

# --- CONFIGURATION ---
SERIAL_PORT = '/dev/ttyUSB0' # À adapter
BAUD_RATE = 115200
TIMEOUT = 0.1
SPEED_TX_PERIOD_S = 0.1
MOTOR_GAIN = 1.65
DRAG_COEFF = 0.9
MODEL_RESPONSE = 1.2
MAX_SPEED = 220.0

# Codes des messages
MSG_SETPOINT = 0x01
MSG_SPEED = 0x02
MSG_MODE_SET = 0x05
MSG_OUTPUT = 0x80
MSG_STATS = 0x83
MSG_ALARM = 0x85

class ECUTester:
    def __init__(self):
        self.ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=TIMEOUT)
        self.current_speed = 50.0 # Vitesse initiale
        self.target_setpoint = 90.0
        self.last_output = 0.0
        self.running = True
        self.stats_count = 0
        self.start_time = time.time()

    def update_vehicle_model(self, dt):
        """Modèle 1er ordre: accel = moteur - traînée."""
        dt = max(0.0, min(dt, 0.2))

```

```

        acceleration = MODEL_RESPONSE * ((MOTOR_GAIN * self.last_output) -
(DRAG_COEFF * self.current_speed))
        self.current_speed += acceleration * dt
        self.current_speed = max(0.0, min(MAX_SPEED, self.current_speed))

def compute_crc(self, data_bytes):
    """Calcule le CRC par XOR de tous les octets sauf le START"""
    crc = 0
    for b in data_bytes:
        crc ^= b
    return crc

def send_frame(self, msg_type, payload=b''):
    """Construit et envoie une trame au format:
[0xAA][LEN][TYPE][PAYLOAD][CRC]"""
    length = len(payload) + 1 # LEN: Taille de (TYPE + PAYLOAD)
    # Format: <H (uint16 little endian), B (uint8)
    header = struct.pack('<HB', length, msg_type)
    full_msg = header + payload
    crc = self.compute_crc(full_msg)
    frame = struct.pack('B', 0xAA) + full_msg + struct.pack('B', crc)
    self.ser.write(frame)

def receive_feedback(self):
    """Lit et décode les messages venant de l'ECU (Output et Télémétrie)"""
    while self.running:
        if self.ser.in_waiting > 0:
            start_byte = self.ser.read(1)
            if start_byte == b'\xaa':
                len_bytes = self.ser.read(2)
                if len(len_bytes) < 2: continue
                length = struct.unpack('<H', len_bytes)[0]

                data = self.ser.read(length)
                crc_received = self.ser.read(1)

                if len(data) == length and crc_received:
                    # Vérification CRC
                    if self.compute_crc(len_bytes + data) == ord(crc_received):
                        msg_type = data[0]
                        payload = data[1:]

                        if msg_type == MSG_OUTPUT:
                            self.last_output = struct.unpack('<f', payload)[0]
                            print(f"[SPEED] ({self.current_speed:.2f} km/h) |
Commande: {self.last_output:.2f}")

                        elif msg_type == MSG_STATS:
                            self.stats_count += 1
                            print(f"[TELEMETRIE] Reçue ({self.stats_count}s)")

```

```

        elif msg_type == MSG_ALARM:
            print(f"\n[ALERTE ECU] :
{payload.decode(errors='ignore')}")

def run_normal_operation(self, duration):
    """Phase de fonctionnement normal (100ms cycle) """
    print(f"--- DÉBUT PHASE NORMALE ({duration}s) ---")
    self.send_frame(MSG_SETPoint, struct.pack('<f', self.target_setpoint))
    self.send_frame(MSG_MODE_SET, struct.pack('B', 2)) # Mode AUTO

    end_time = time.time() + duration
    last_tick = time.time()
    while time.time() < end_time:
        now = time.time()
        dt = now - last_tick
        last_tick = now

        self.update_vehicle_model(dt)
        # Envoi de la vitesse mesurée toutes les 100ms
        self.send_frame(MSG_SPEED, struct.pack('<f', self.current_speed))
        time.sleep(SPEED_TX_PERIOD_S)

def run_stress_test(self):
    """Phase de stress : messages corrompus, fragmentés et flood"""
    print("\n--- DÉBUT PHASE DE STRESS ---")

    # 1. Flood de messages (Surcharge CPU)
    print("Action: Surcharge CPU (Flood)...")
    for _ in range(50):
        self.send_frame(random.randint(0, 0xFF), os.urandom(4))

    # 2. Trames fragmentées
    print("Action: Envoi de trame fragmentée...")
    partial_frame = struct.pack('B', 0xAA) + struct.pack('<HB', 5, MSG_SPEED)
    self.ser.write(partial_frame)
    time.sleep(0.5) # Pause au milieu du message
    self.ser.write(struct.pack('<f', 100.0) + b'\x00') # Fin de trame

    # 3. Erreur de CRC
    print("Action: Envoi erreur CRC...")
    self.ser.write(b'\xAA\x05\x00\x02\x00\x00\x00\x00\xFF')

def test_failsafe(self):
    """Vérifie si l'ECU passe en Failsafe après 2s de silence [cite: 53]"""
    print("\n--- TEST FAILSAFE (Silence radio 2s) ---")
    time.sleep(2.5)
    if abs(self.last_output) < 1e-3:
        print("VÉRIFICATION RÉUSSIE : L'ECU a coupé la commande moteur
(Output=0)")

```

```
    else:
        print("ÉCHEC : L'ECU n'est pas passé en mode Failsafe")

def start(self):
    # Lancement du thread de lecture
    reader = threading.Thread(target=self.receive_feedback, daemon=True)
    reader.start()

    try:
        self.run_normal_operation(30) # 30s de fonctionnement normal
        self.run_stress_test()
        self.run_normal_operation(5)
        self.test_failsafe()
    except KeyboardInterrupt:
        pass
    finally:
        self.running = False
        self.ser.close()
        print("\nTest terminé.")

if __name__ == "__main__":
    tester = ECUTester()
    tester.start()
```